

Import math

```
Board = [  
    ['P', '.', '.', '.'],  
    [., '#', '.', '.'],  
    [., '.', '#', '.'],  
    [., '.', '.', 'G']  
]
```

```
Moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
Player = [0, 0]
```

```
AI = [3, 3]
```

```
Traps = {}
```

```
Def show():
```

```
    For r in range(4):
```

```
        For c in range(4):
```

```
            If [r, c] == player:
```

```
                Print('P', end=' ')
```

```
            Elif [r, c] == ai:
```

```
                Print('A', end=' ')
```

```
            Elif (r, c) in traps:
```

```
                Print('T', end=' ')
```

```
            Else:
```

```
Print(board[r][c], end='')
```

```
Print()
```

```
Print()
```

```
Def valid(r, c):
```

```
Return 0 <= r < 4 and 0 <= c < 4 and board[r][c] != '#'
```

```
Def distance(a, b):
```

```
Return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
Def minimax(pos, player_pos, depth, alpha, beta, maximizing):
```

```
If pos == player_pos:
```

```
Return 50
```

```
If pos == [0, 0]:
```

```
Return -50
```

```
If depth == 0:
```

```
Return -distance(pos, [0, 0])
```

```
If maximizing:
```

```
Best = -math.inf
```

For dr, dc in moves:

R = pos[0] + dr

C = pos[1] + dc

If valid(r, c):

Value = minimax(  
    [r, c], player\_pos,  
    Depth - 1, alpha, beta, False  
)

If (r, c) in traps:

Value -= 20

Best = max(best, value)

Alpha = max(alpha, best)

If beta <= alpha:

Break

Return best

Else:

Best = math.inf

For dr, dc in moves:

R = player\_pos[0] + dr

```
C = player_pos[1] + dc
```

```
If valid(r, c):
```

```
    Value = minimax(
```

```
        Pos, [r, c],
```

```
        Depth - 1, alpha, beta, True
```

```
    )
```

```
Best = min(best, value)
```

```
Beta = min(beta, best)
```

```
If beta <= alpha:
```

```
    Break
```

```
Return best
```

```
Def ai_move():
```

```
    Global ai
```

```
    Best_value = -math.inf
```

```
    Best_move = ai
```

```
    For dr, dc in moves:
```

```
        R = ai[0] + dr
```

```
        C = ai[1] + dc
```

If valid(r, c):

```
Value = minimax(  
    [r, c], player,  
    3, -math.inf, math.inf, False  
)
```

If (r, c) in traps:

```
Value -= 20
```

If value > best\_value:

```
Best_value = value
```

```
Best_move = [r, c]
```

Ai = best\_move

Def update\_traps():

```
Expired = []
```

For pos in traps:

```
Traps[pos] -= 1
```

If traps[pos] <= 0:

```
Expired.append(pos)
```

For pos in expired:

Del traps[pos]

While True:

Show()

If player == [3, 3]:

Print("Player reached the goal!")

Break

If ai == player:

Print("AI caught the player!")

Break

Choice = input("W/A/S/D to move, T to place trap: ").lower()

If choice == 't':

Traps[tuple(player)] = 2

Print("Trap placed!")

Update\_traps()

Ai\_move()

Continue

Direction = {

```
    'w': (-1, 0),  
    's': (1, 0),  
    'a': (0, -1),  
    'd': (0, 1)  
}
```

If choice not in direction:

Print("Invalid move!")

Continue

Dr, dc = direction[choice]

R = player[0] + dr

C = player[1] + dc

If valid(r, c):

Player = [r, c]

If (r, c) in traps:

Print("💣 You stepped on a trap!")

Break

Update\_traps()

Ai\_move()